

Response to Reviewer 1

April 29, 2026 · Manuscript ID: scCCVGBen-2026

We thank Reviewer 1 for the detailed and constructive feedback. Below we address each point in order and report only manuscript changes or completed checks that are included in the submission package.

R1.1 — Dataset information and summary table

Reviewer comment

The claim of evaluating 170 datasets (55 scRNA-seq and 115 scATAC-seq) is impressive, but the dataset information is currently unclear. Important details are missing, such as the number of cells per dataset, the preprocessing pipeline used, and whether baseline methods were tuned or run with default settings. Without this information, it is difficult to properly interpret the benchmarking results. It would be helpful to include a summary table of all datasets, listing information such as dataset name, modality, number of cells, number of genes/features, source repository, and dataset link. This table could be placed in the main text or supplementary materials, in addition to the links provided in the Data Availability section.

Authors' response

We agree that the dataset details were insufficient. In the revised manuscript the cohort has grown from 170 to 200 datasets (100 scRNA-seq and 100 scATAC-seq). We have added Supplementary Table S1, which lists every dataset with the following fields: dataset name, modality, number of cells, number of genes or peaks (as appropriate), source repository, GEO/ENCODE accession, and a direct download link. The table is generated programmatically from the released benchmark manifest so that every column is traceable and reproducible. Regarding tuning: all baseline methods were run with their published default hyperparameters; no dataset-specific tuning was performed. This is now stated explicitly in the Methods section.

Revised manuscript text

See Supplementary Table S1 (built from benchmark manifest) listing all 200 datasets with name, modality, n_{cells} , n_{features} , source, accession, and link. A note on default-hyperparameter baselines has been added to the Methods (Section 2, “Baseline methods”).

R1.2 — Clustering algorithm and parameter specification

Reviewer comment

It is unclear how the clusters were generated in the downstream analysis. The authors should specify which clustering algorithm was used (e.g., Leiden, Louvain, or k-means) and the key parameters. In addition, when clustering is performed on embeddings with different geometries, the results may depend strongly on the clustering parameters rather than the quality of the embedding itself. Thus, more details about the clustering procedure are needed to properly interpret the reported metrics.

Authors' response

We used Leiden community detection (Traag et al., 2019) (resolution 1.0, modularity-vertex partition) with a fixed integer seed (42) propagated through all training and evaluation calls. This is now stated in a dedicated paragraph in the Methods (Section 2, “Clustering”). To address the concern about parameter sensitivity we ran two additional experiments (D5 and D6):

- **D5** (clustering-method sensitivity): we re-ran the same latent representations through k-means and Louvain with the same resolution, and report ARI, NMI, ASW, DAV, and CAL for all three algorithms across all 200 datasets. Results are in Supplementary Fig. S2.
- **D6** (multi-seed clustering ARI): we repeated the Leiden partitioning five times per (dataset, method) pair with different random seeds and report median ARI $\pm$ IQR. Results are in Supplementary Fig. S3.

Both experiments confirm that ranking stability across methods is preserved regardless of the clustering choice.

Revised manuscript text

New Methods paragraph (Section 2, “Clustering”): “Cluster labels were generated with the Leiden algorithm at resolution 1.0 using the modularity-vertex partition and a fixed seed of 42; sensitivity to algorithm and seed is reported in Supplementary Figs. S2 and S3.”

R1.3 — Empirical validation of embedding stability**Reviewer comment**

The authors claim that stochastic sampling in VAEs degrades stability, reproducibility, and the geometric integrity of the latent space. They further state that using the posterior mean as a deterministic centroid embedding provides stable inference by removing sampling variability. However, the experiments in the manuscript do not directly evaluate embedding stability. To support this claim, the authors should demonstrate that centroid inference reduces variability in the embeddings, for example, by running the model multiple times with different random seeds and measuring the variance of the resulting embeddings or clustering outcomes.

Authors' response

The reviewer is correct that the original stability wording was too broad without a direct reproducibility check. In the revised manuscript we narrow the claim to the deterministic inference step and add a completed multi-seed sanity check for the centroid configuration. The seed is propagated to the cell-subsample RNG, NumPy, and the PyTorch CPU/CUDA random states so that cell subsetting, subgraph sampling order, and model initialisation vary jointly.

Completed centroid-configuration summary from the available multi-seed runs:

Variant	ASW (mean)	SD _{ASW} across seeds	Overall UMAP (mean)
Centroid (scCCVGBen)	0.432	0.031	0.737

The observed ASW standard deviation (0.031) shows that the centroid configuration is reproducible under the evaluated seed perturbations; a typical seed change perturbs ASW by approximately 7% of the mean. The theoretical argument is also stated explicitly: using the posterior mean removes the Monte-Carlo sampling term from inference, so the reported embedding is not a single random latent draw.

Revised manuscript text

New subsection “Stability of the centroid embedding” added after Section 3.1. The revised text reports the completed centroid multi-seed check and narrows the claim to deterministic posterior-mean inference rather than an unsupported comparison against unreported stochastic variants.

R1.4 — Minimal example dataset on GitHub**Reviewer comment**

Please provide a minimal example dataset on GitHub to help users reproduce the analysis pipeline.

Authors' response

We have added a minimal example to the public code repository (<https://github.com/PeterPonyu/scCCVGBen>). The example is a self-contained 30-line Jupyter notebook that loads the 2 700-cell PBMC dataset bundled with the Scanpy library (Wolf et al., 2018) (no separate download required), trains scCCVGBen for 5 epochs, and renders a UMAP coloured by the predicted cluster labels. The notebook depends only on Scanpy, PyTorch, and the scCCVGBen package itself. Runtime on CPU is under 3 minutes. A deployed companion site at <https://peterponyu.github.io/scccvgben-next/> additionally hosts per-dataset metadata cards and an interactive view of the 200-dataset cohort, so a reviewer can browse cohort composition without cloning the repository.

Revised manuscript text

Minimal example notebook added to the public code repository, and a deployed companion site provides a clone-free view of the cohort — both links are now listed in the Code Availability section of the revised manuscript.